Now we want the `new` action to present users with a form, prompting them to enter the information in the `orders` table: the user's name, address, email address, and payment type. This means we'll need to display a Rails template containing a form. The input fields on this form will have to link to the corresponding attributes in a Rails model object, so we need to create an empty model object in the `new` action to give these fields something to work with.

As always with HTML forms, the trick is populating any initial values into the form fields and then extracting those values out into our application when the user clicks the submit button.

In the controller, the `order` variable is set to reference a new `Order` model object. This is done because the view populates the form from the data in this object. As it stands, that's not particularly interesting. Because it's a new model object, all the fields will be empty. However, consider the general case. Maybe we want to edit an existing order. Or maybe the user has tried to enter an order but the data has failed validation. In these cases, we want any existing data in the model shown to the user when the form is displayed. Passing in the empty model object at this stage makes all these cases consistent. The view can always assume it has a model object available. Then, when the user clicks the submit button, we'd like the new data from the form to be extracted into a model object back in the controller.

Fortunately, Rails makes this relatively painless. It provides us with a bunch of *form helper* methods. These helpers interact with the controller and with the models to implement an integrated solution for form handling. Before we start on our final form, let's look at a small example:

```
<%= form_with(model: order) do |form| %>
  <p>
    <%= form.label :name, "Name:" %>
    <%= form.text_field :name, size: 40 %>
  </p>
<% end %>
```